实验目的：

熟悉并掌握 Wireshark 的基本操作，了解网络协议实体间进行交互以及报文交换的情况。

实验内容：

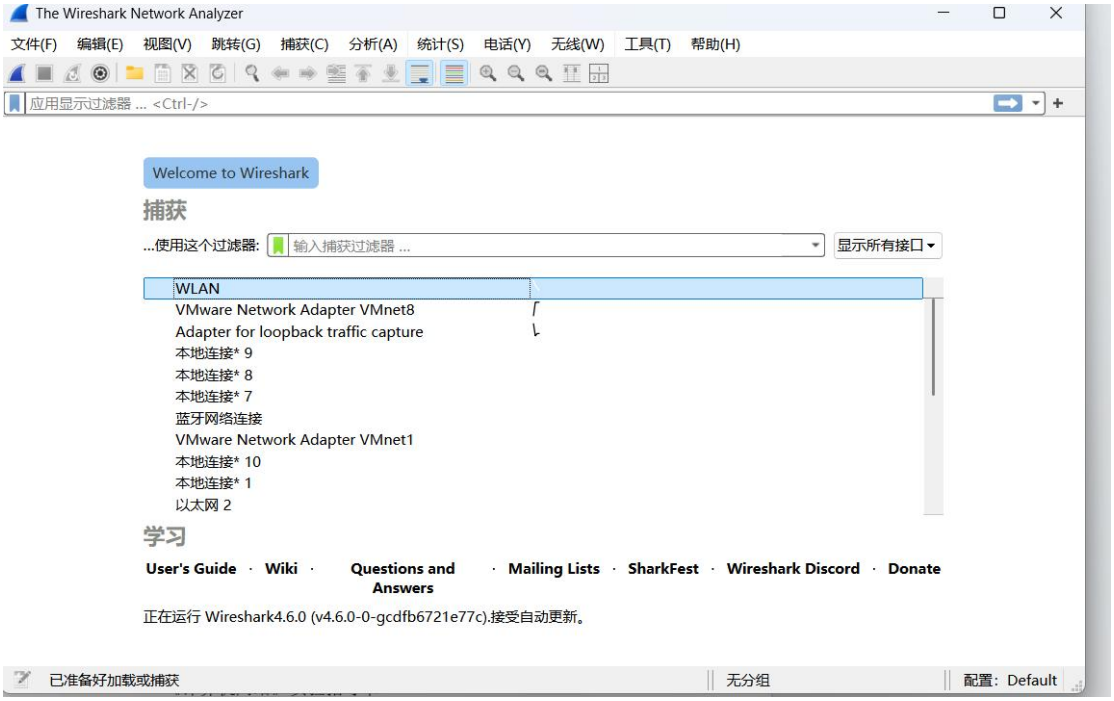
1) 学习 Wireshark 的使用
2) 利用 Wireshark 分析 HTTP 协议
3) 利用 Wireshark 分析 TCP 协议
4) 利用 Wireshark 分析 IP 协议
5) 利用 Wireshark 分析 Ethernet 数据帧

选做内容：

a) 利用 Wireshark 分析 DNS 协议
b) 利用 Wireshark 分析 UDP 协议
c) 利用 Wireshark 分析 ARP 协议

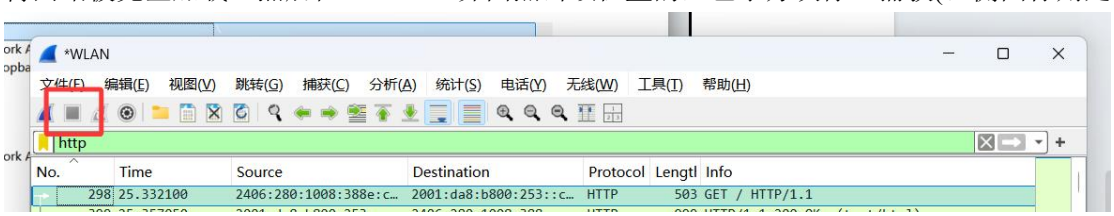
实验过程：

（一）Wireshark 的使用

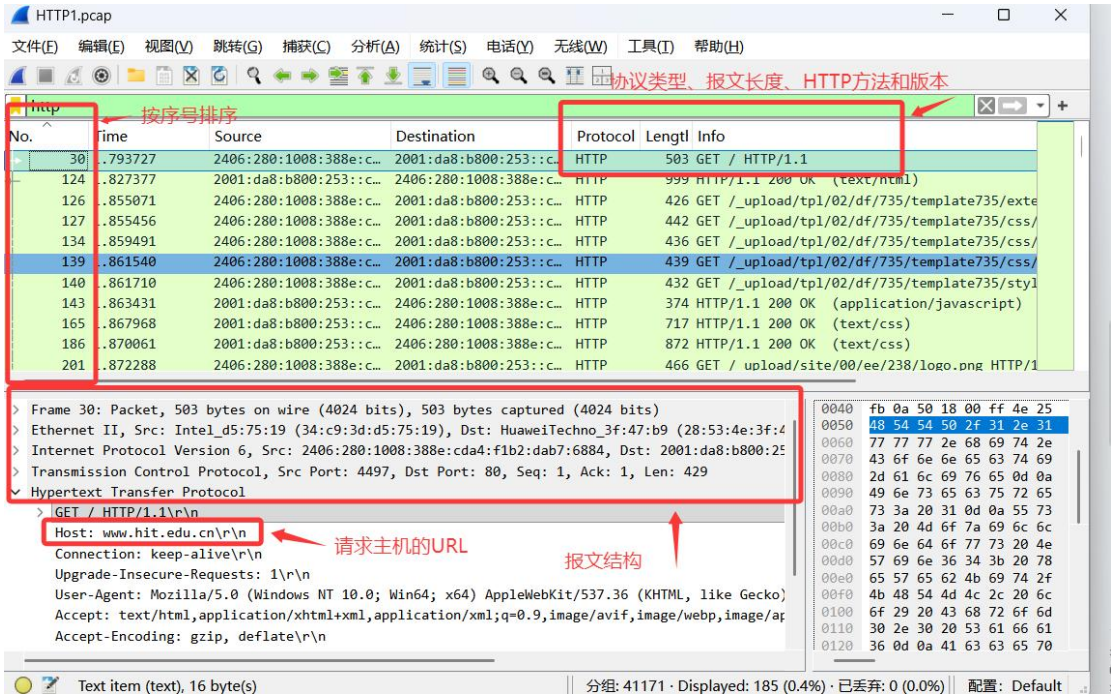


Wireshark安装后，打开程序，程序会自动检测设备具有的网卡信息，选择你需要监听的网卡(由于本次实验再WLAN环境下进行，因此选择WLAN开始监听)。

开始监听后，在浏览器中输入工大官网的网址https://www.hit.edu.cn，随后下滑和等待，使得网站被完全加载。然后在Wireshark界面点击该位置的红色小方块停止捕获(左侧图标则是



重新启动捕获)。停止捕获后，在下方过滤规则烂输入http，Wireshark就会自动过滤选择出使用HTTP协议的报文，由于整个过程只捕捉了官网，因此涉及到的HTTP的报文都是与此相关的，选择按Wireshark分配的序号或者时间排序皆可(降序)，第一条报文即为HTTP请求报文。



整体的框架，从上到下依次为：

（1）Frame：物理层的数据帧概况

（2）Ethernet II：数据链路层以太网帧头部信息

（3）Internet Protocol Version 4：互联网层IP包头部信息

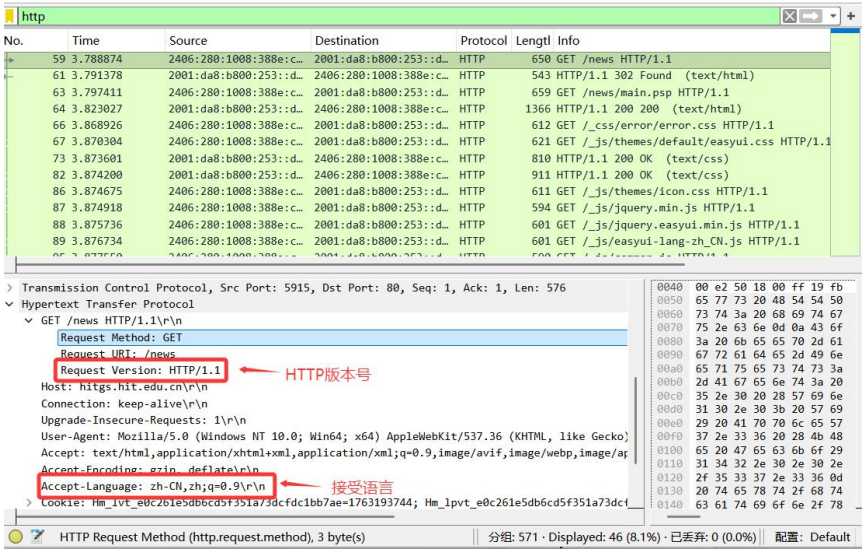
（4）Transmission Control Protocol：传输层的数据段头部信息，此处是TCP

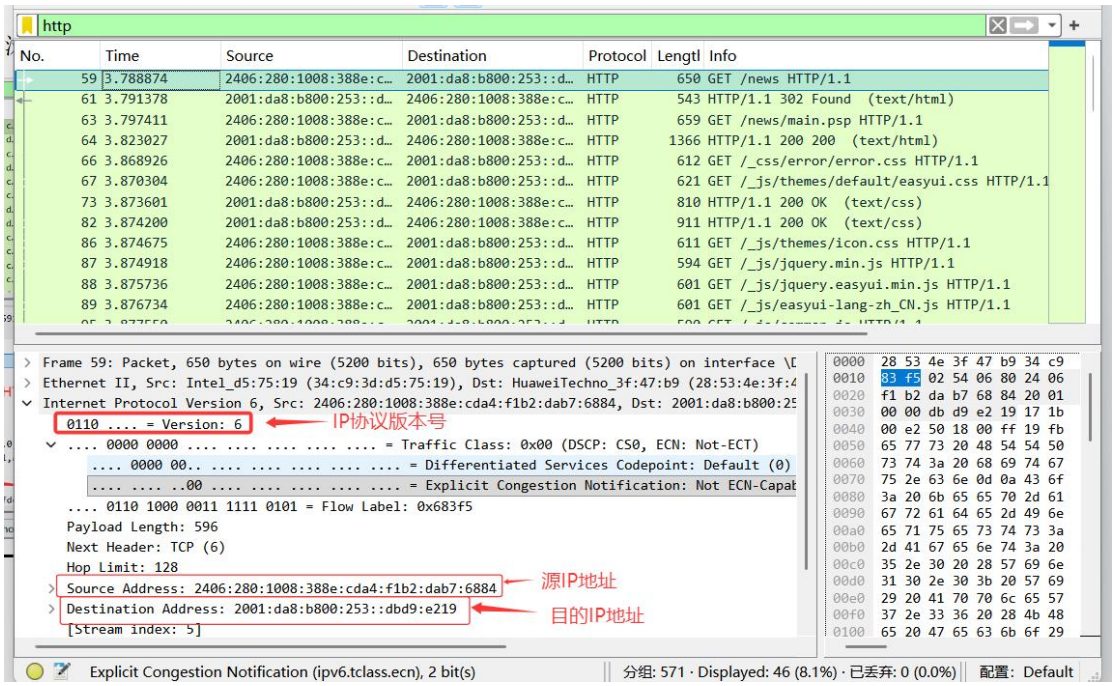
（5）Hypertext Transfer Protocol：应用层的信息，此处为HTTP协议。

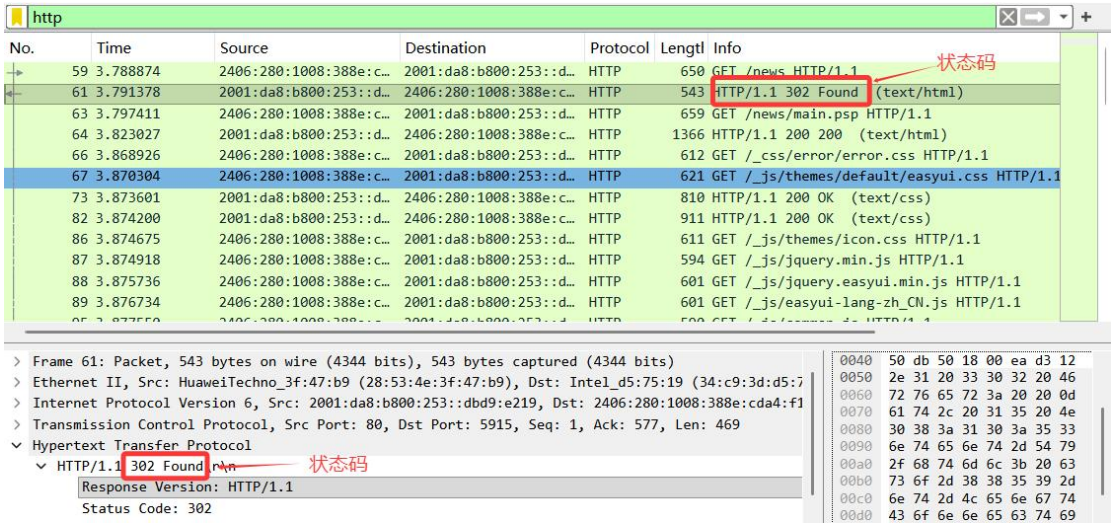
（二）HTTP 分析

1）HTTP GET/response 交互

Accept-Language行表示浏览器接受的语言，GET方法行最后包含了HTTP协议的版本。







状态码则可以在捕获的数据报的Info栏目或者下方报文中的HTTP报文详细信息中看到，为302，表示请求的资源临时从不同的URI响应请求(资源已经临时不在这个URL)。

2）HTTP 条件 GET/response 交互

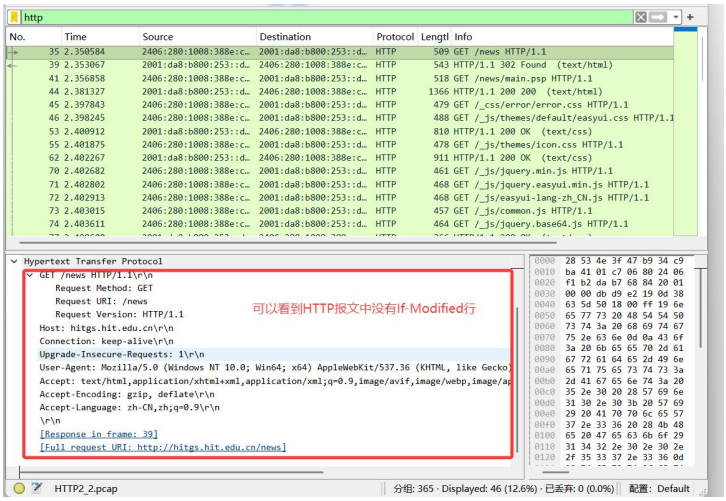
本次实验过程中使用了Chrome浏览器进行。为了清除浏览数据，我们需要点击浏览器右上角三点号，随后在下方找到删除浏览数据一行，点击该行。会自动跳转至对应的设置页面。



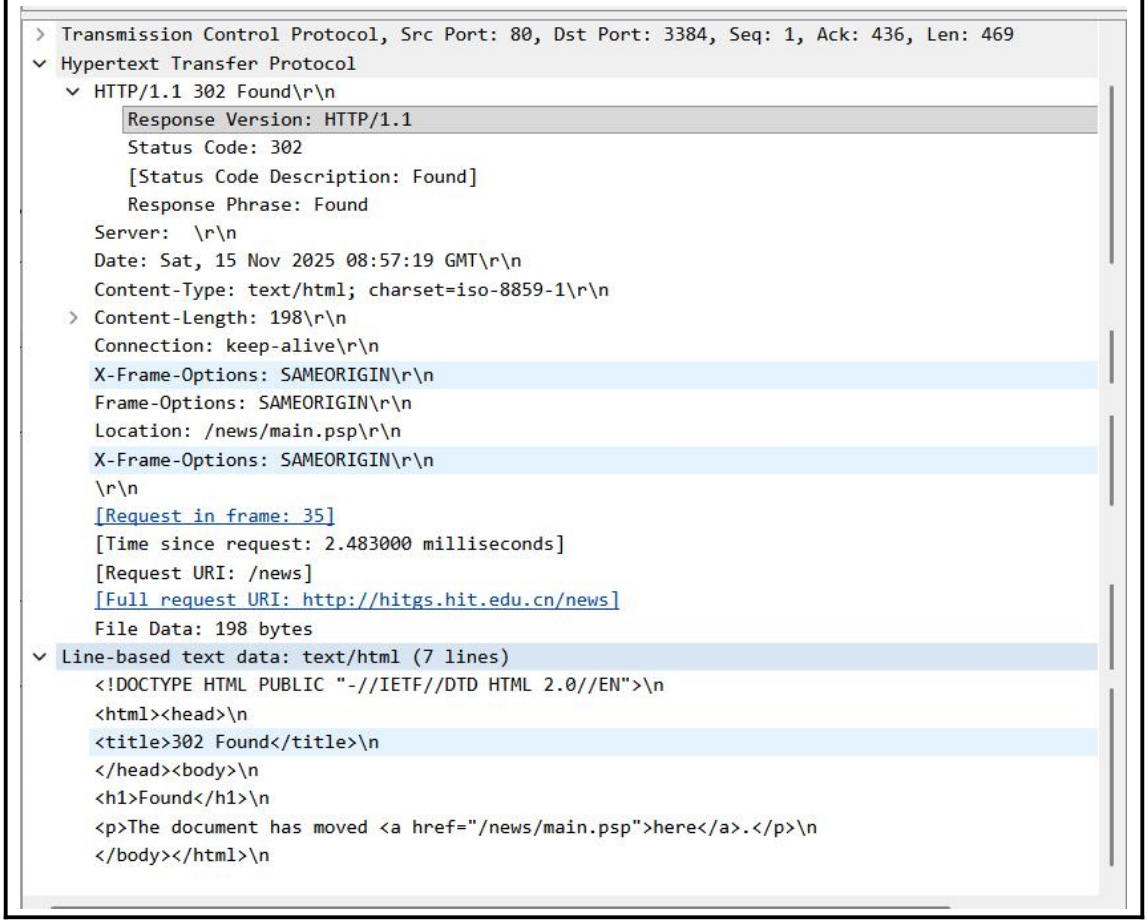
点击下方的删除数据即可。



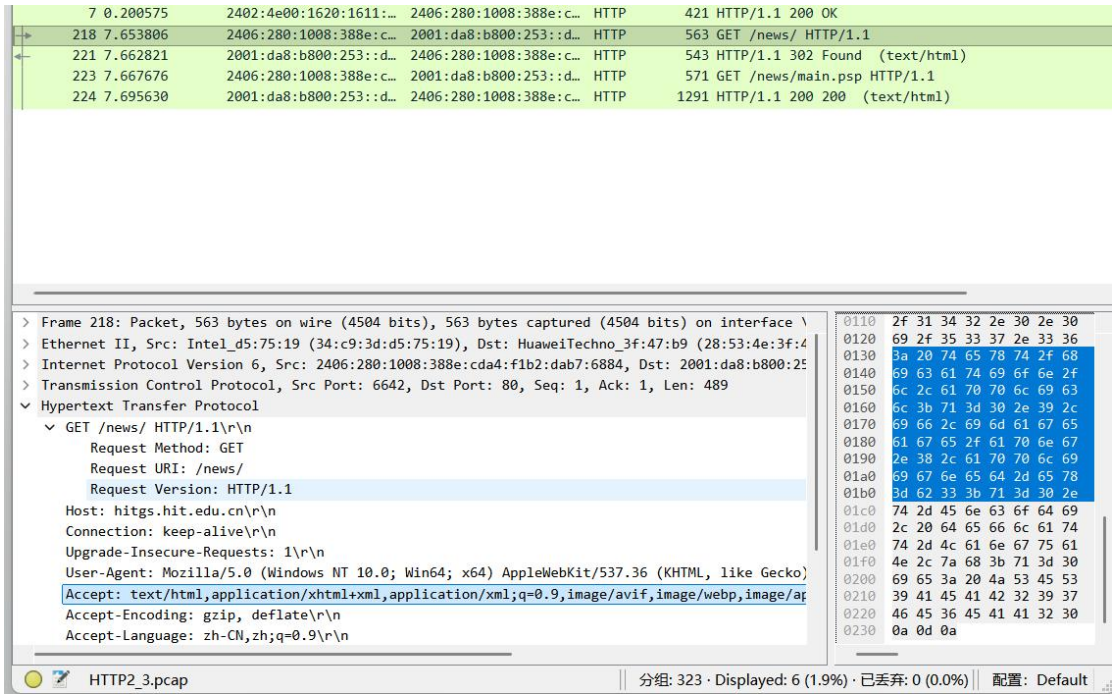
此时我们重新访问网站，可以看到没有 If-Modified 行



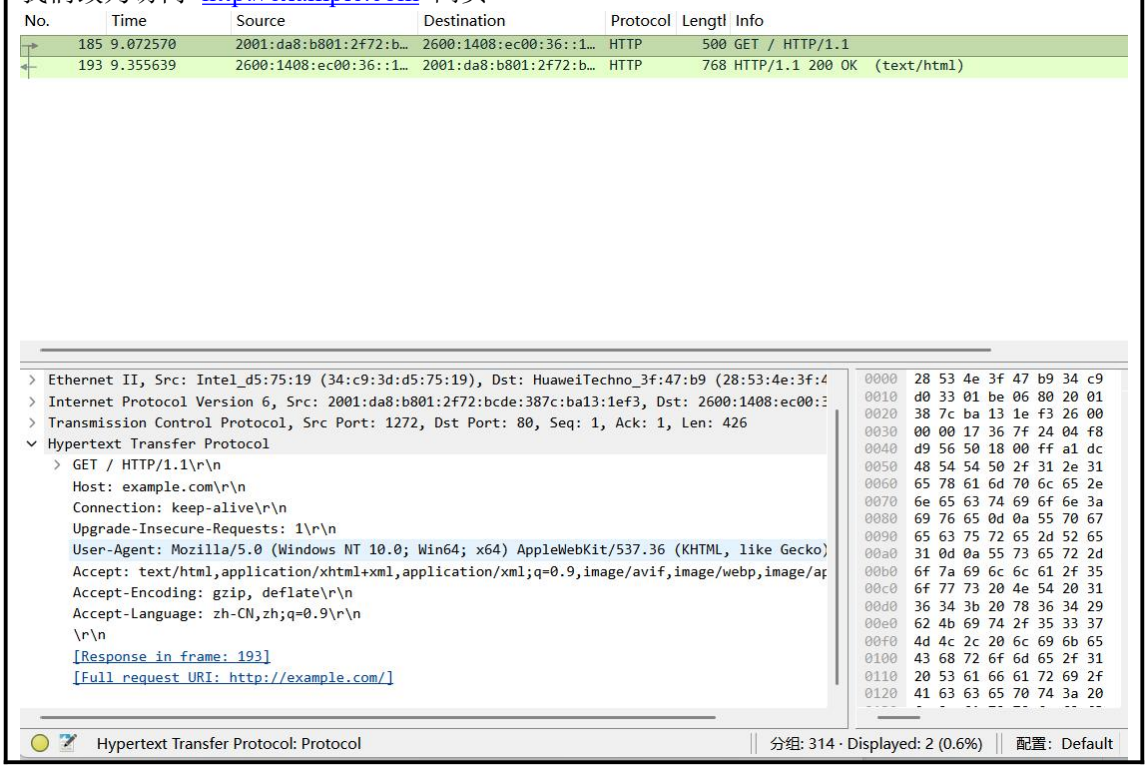
报文内容为使用 Get 方法获取/news，使用协议为 HTTP1.1。请求的主机名为 hitgs.hit.edu.cn。连接方式为保持持续连接。Upgrade-Insecure-Requests 值是固定为"1"，表示客户端愿意优先使用 HTTPS。Accept 行告诉服务器客户端能够处理的内容类型，包括 text、html、xhtml、image、webf 类型等。



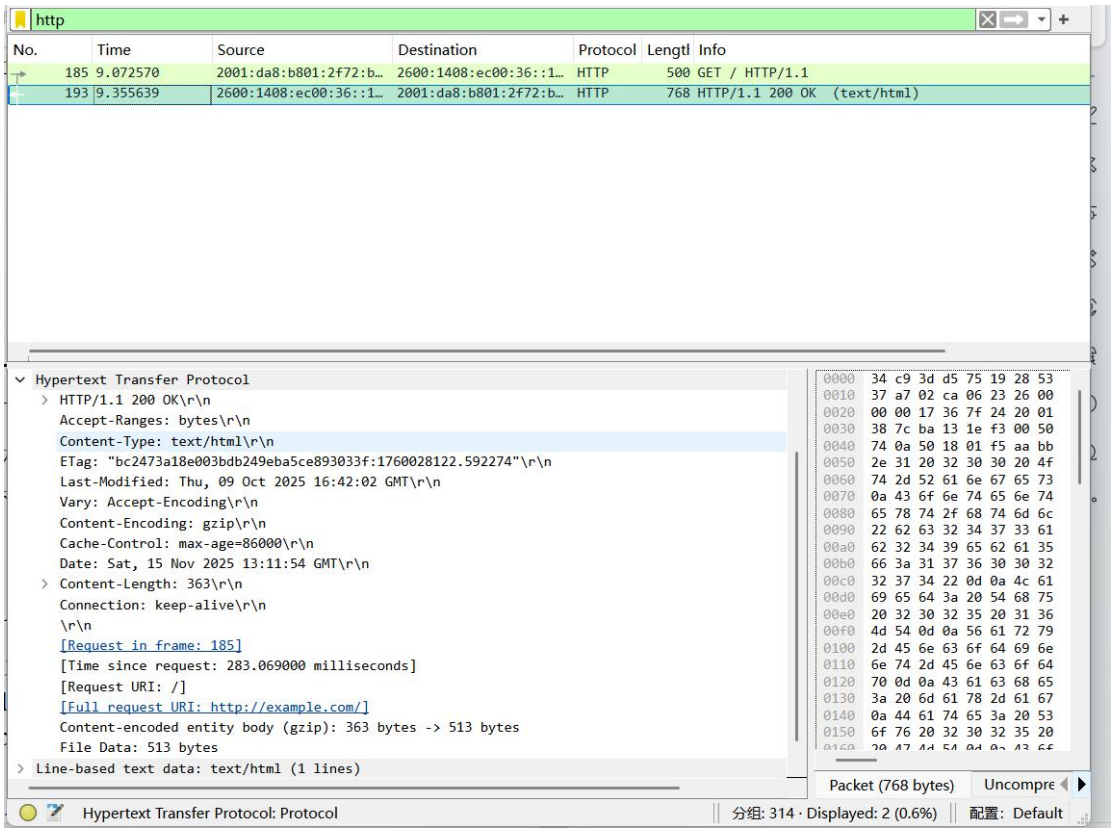
Accept-Encoding 告诉服务器客户端支持的压缩编码，在这个报文中为 gzip、deflate。Accept-Language 告诉服务器客户端期望看到的语言。 User-Agent 行标识客户端软件信息。上页最后一张图片为删除浏览器数据后第一次重新访问的 HTTP 响应报文。由图可知，在第一次访问时，HTTP 响应报文中含有 Data 行，表示了数据修改时间、数据长度、数据类型等信息，在 HTTP 响应报文下，还可以看到存在 Line-based text data 层，即说明存在数据，展开 Line-based text data 层就可以看到传输的内容。



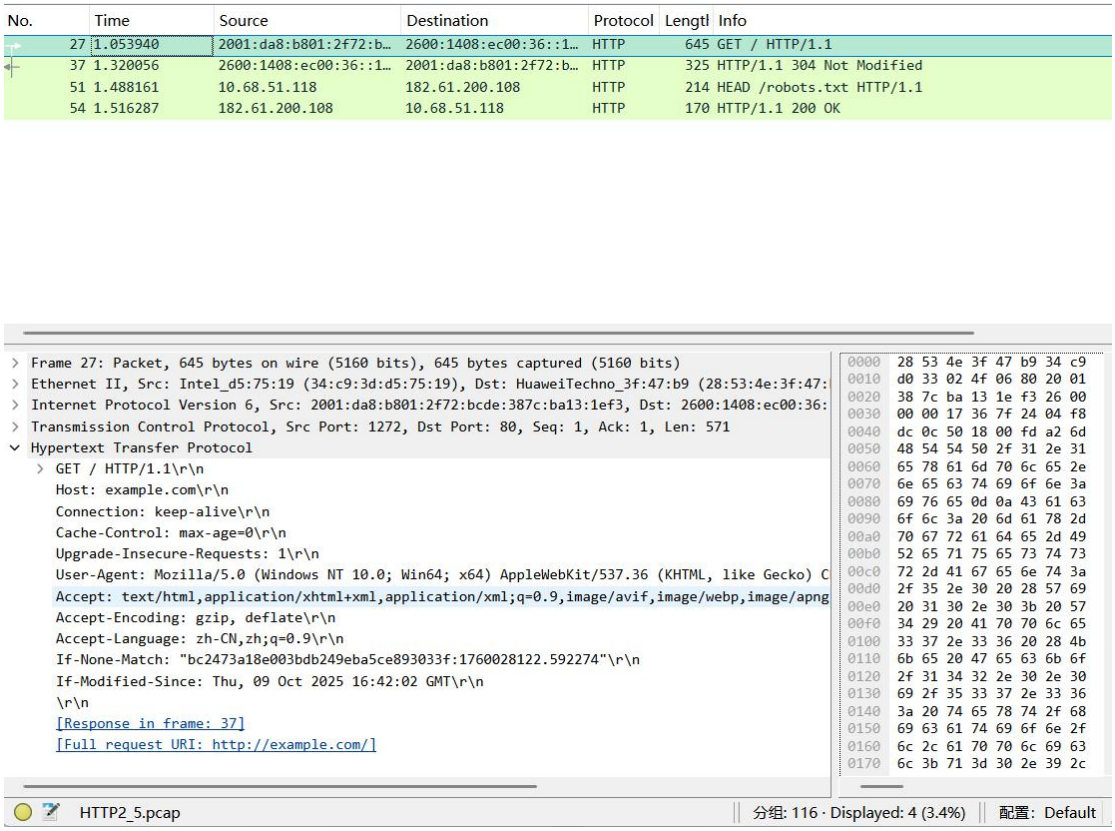
我们改为访问 http://example.com 网页



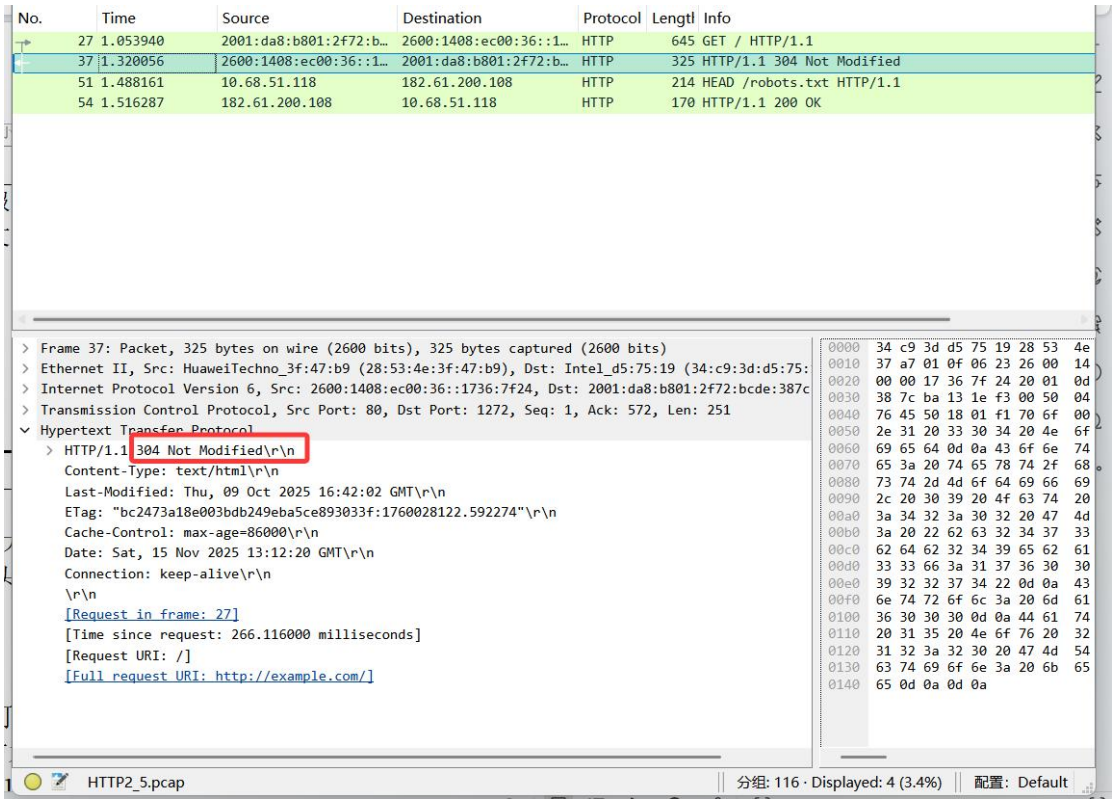
可以看到在第一个HTTP请求报文也不存在If-modified-since行，其余内容除了主机名外基本内容相同。而在响应报文中，也携带了text/html文件。但是响应报文中出现了Last-modified行



重新刷新访问http://example.com

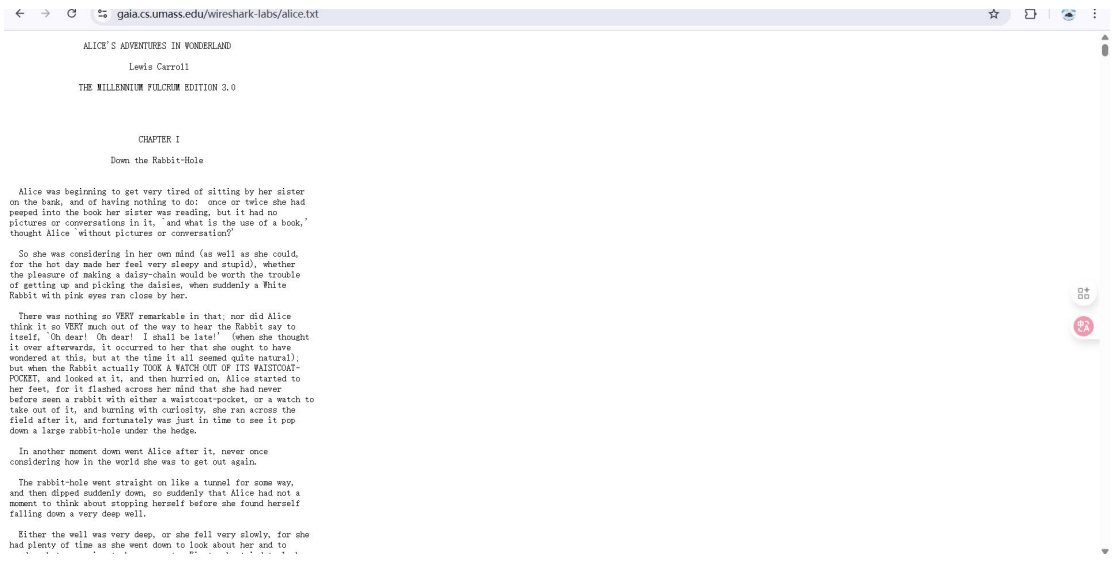


可以看到，重新访问的HTTP报文中存在If-modified-since行，其后的时间戳是清除浏览数据后第一次访问时HTTP响应报文中的Last-modified行的时间戳。除此之外，返回的状态码为304 Not Modified。而且之后与第一次访问相比没有文件内容(Line-based text data)。



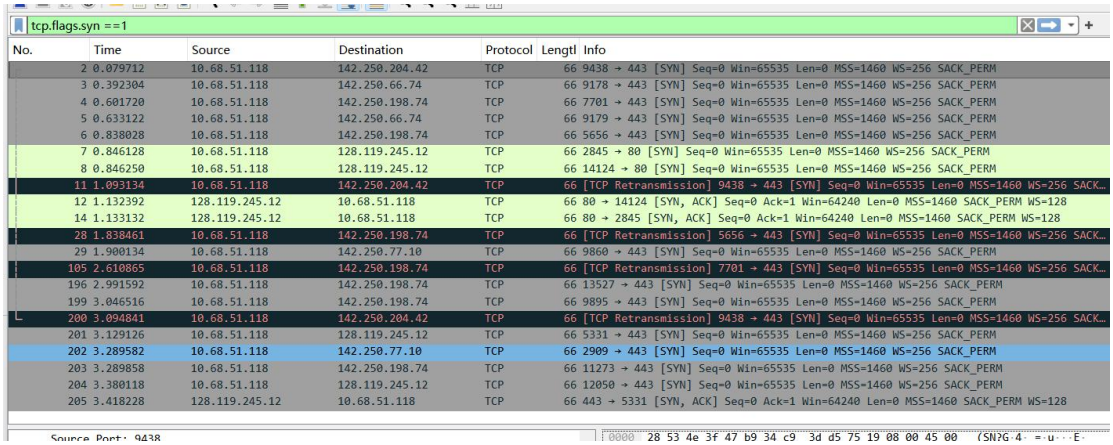
（三）TCP 分析

首先先来到给定的网站，将网站内容复制后保存到 alice.txt 中，在此之后，继续访问 https://gaia.cs.umass.edu/wireshark-labs/TCP-wireshark-file1.html 点击选择文件，在保存的路径下找到 alice.txt 文件并点击打开。



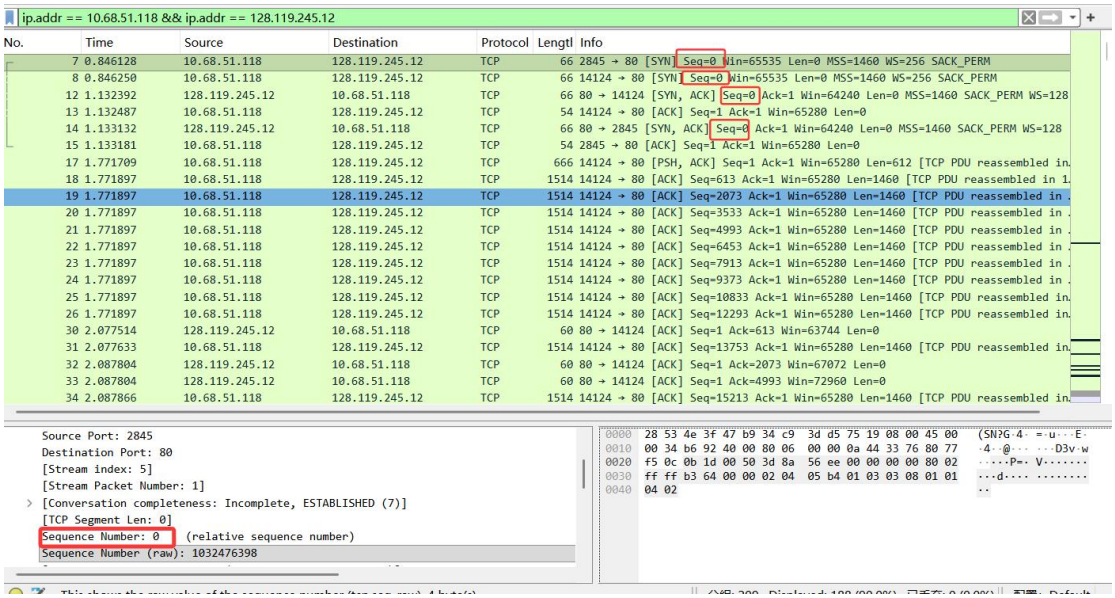
此时打开 Wireshark 先开始捕获，然后点击 upload alice.txt file 按钮点击开始上传 alice.txt 文件，出现上传成功的网页后立马停止捕获。

由于这个过程中存在其他 TCP 连接干扰而存在多次握手，为了快速定位报文确认因此先选择向过滤器规则中填入 tcp.flags.syn ==1，表示筛选 syn 位置为 1 的 tcp 报文，

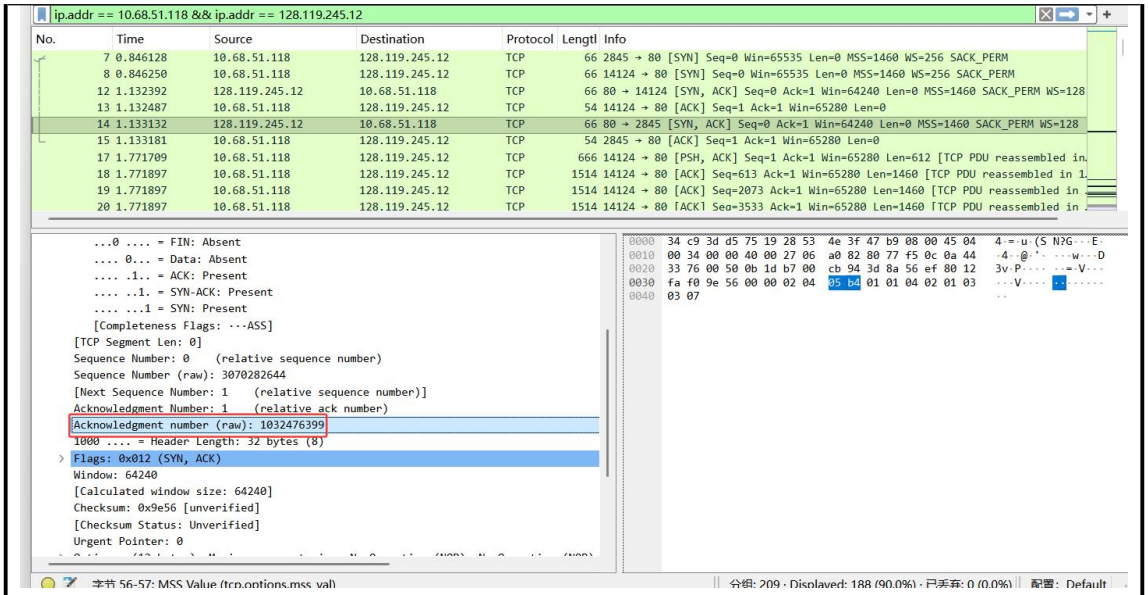


排除掉使用 IPv6 的网址，仅有 SYN 置为 1 的报文即为客户端的报文，而 SYN、ACK 位均置 1 的报文为相应的握手的回复报文。存在 TCP 握手报文的 IP 地址为 128.199.245.12、142.250.196.206 等，但除了 128.199.245.12 外都没有回复握手报文，只建立了半开连接。因此确认 128.199.245.12 即为 gaia.cs.umass.edu 服务器的 IP 地址，进一步从 info 中可见发送端口号为 2845 或者 14124，接收端口号为 80。

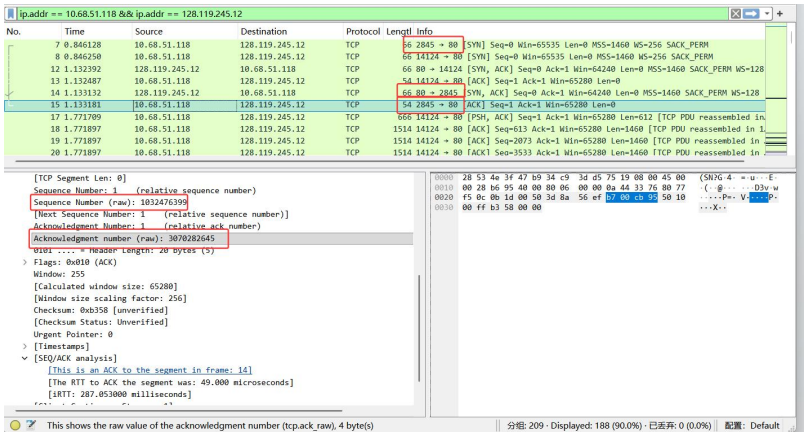
选择客户端 2845 端口对应的 TCP 数据包可见相对 Seq 为 0，实际的 Seq 则为其下方的 1032476398



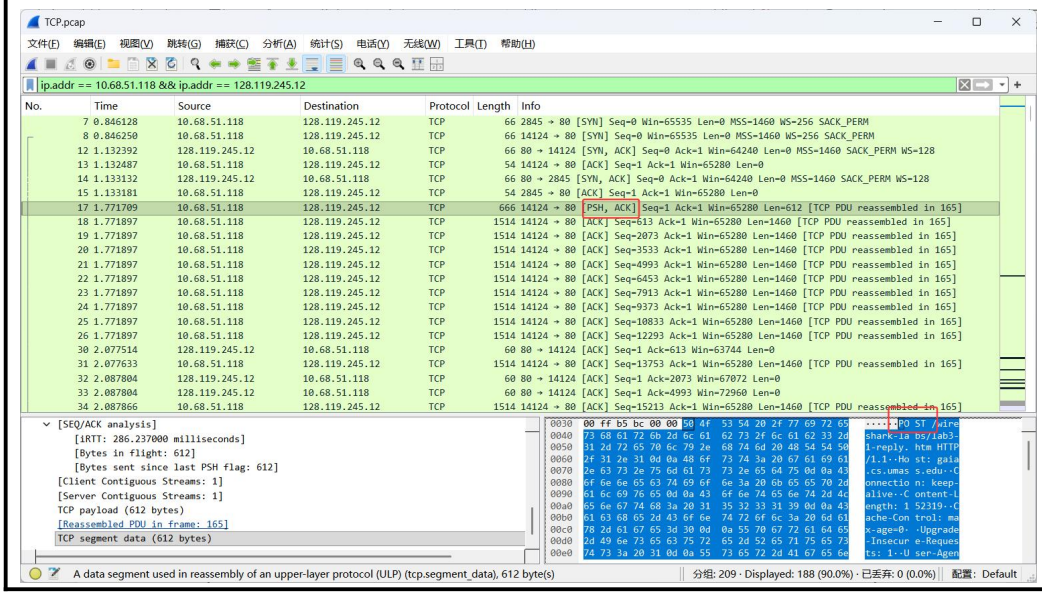
服务器端发送的 SYNACK 报文 Seq 序号也为 0(实际为 307028264)，而 ACK 为 1(实际为 1032476399)，与上面的 Seq 号相对应。在该报文段中，正是因为首部中的 SYN、ACK 标志位都置为 1，且实际的 Seq 号与 ACK 号相对应，所以才能确定这是第二次握手的报文。



再看 2845 端口发送的第二次报文，可以看到实际的 Seq 号为 1032476399、ACK 号为 3070282645，与上次服务器发送的 ACK 号、Seq 号相吻合，该报文将被发送给服务器端用于第三次握手，该报文中可以承载信息。服务器端收到该报文后建立连接。至此，已经建立了完整的 TCP 连接。

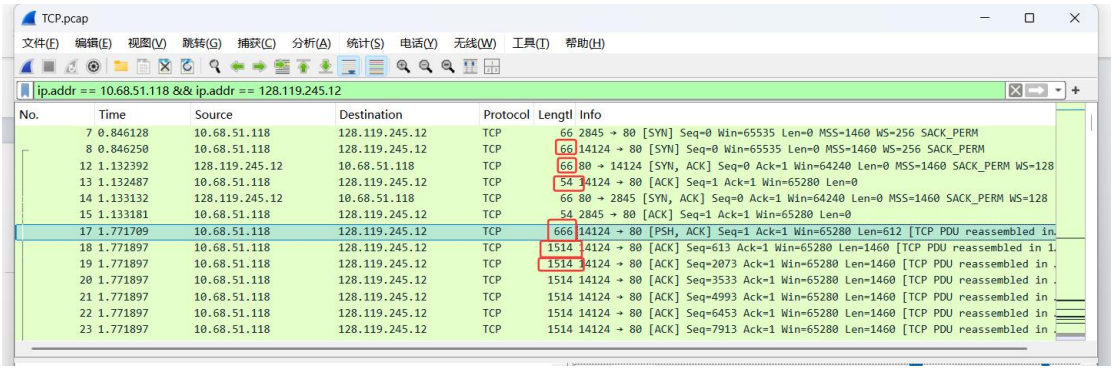


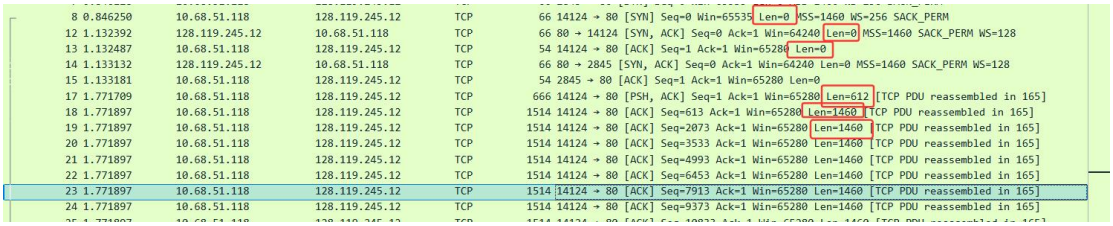
POST 是 HTTP 协议中用于向服务器提交数据的请求方法，主要用于创建新资源或处理数



据。因此包含 HTTP POST 命令的 TCP 报文段应当是 PSH 标识被设置为 1 的 TCP 段，如下图所示，因为这是在建立(与 14124 端口)TCP 连接后，客户端向服务器段发送的第一个 TCP 数据段。

后续由于只使用 14124 端口，因此以 14124 端口继续分析。该端口从向服务器发送 TCP 连接请求开始，发送的报文段长度依次为 66、66、54、666、1514、1514，数据量则依次为

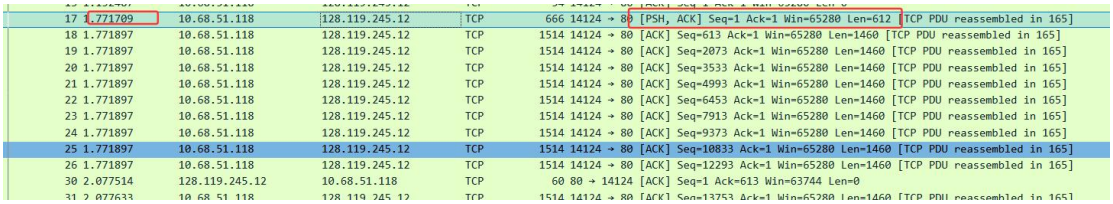




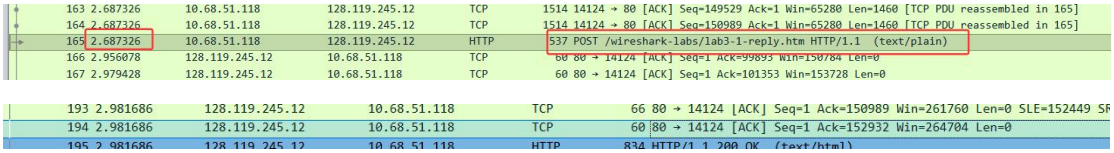
0、0、0、612、1460、1460。

浏览全部数据包中的 info 列，找到其中最小的 info 行，即为最小的接收窗口，为 63744。缓存大小存在一定的变化。最常见的大小为 65280。

虽然存在重复 ACK 确认的出现，但浏览全部由 14124 端口发送/接收的报文，可以看到 Seq 号是单调递增且无重复的。只有 85293、113033、122601、136065、138985、143365 几个数据包被发送了重复确认。



从第一次 POST 开始传输数据，此时时间为 1.771709，直到最后一个数据(reply.htm)报传输完毕时，时间为 2.687326。最后一个 ACK 号为 152932。故累计传输数据的字节数为 152932B

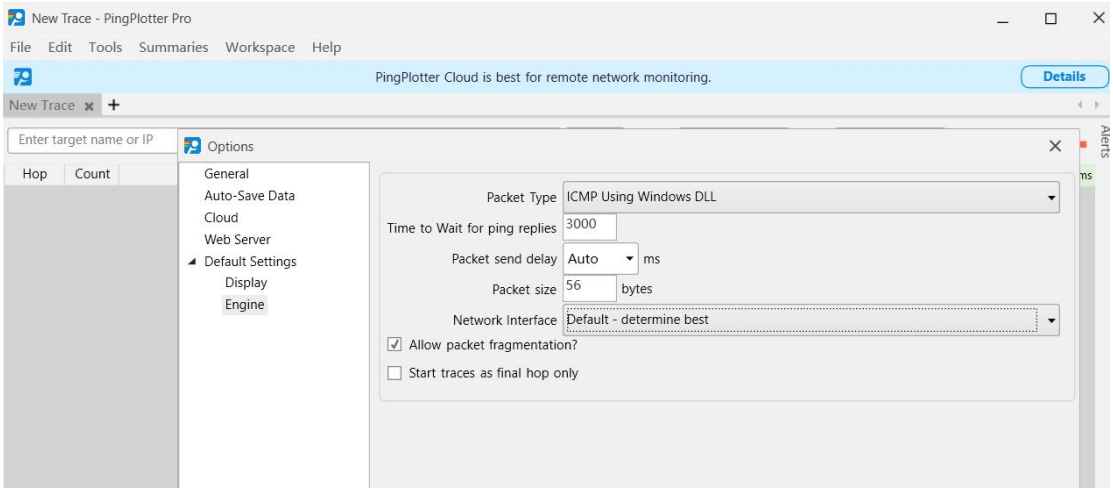


因此

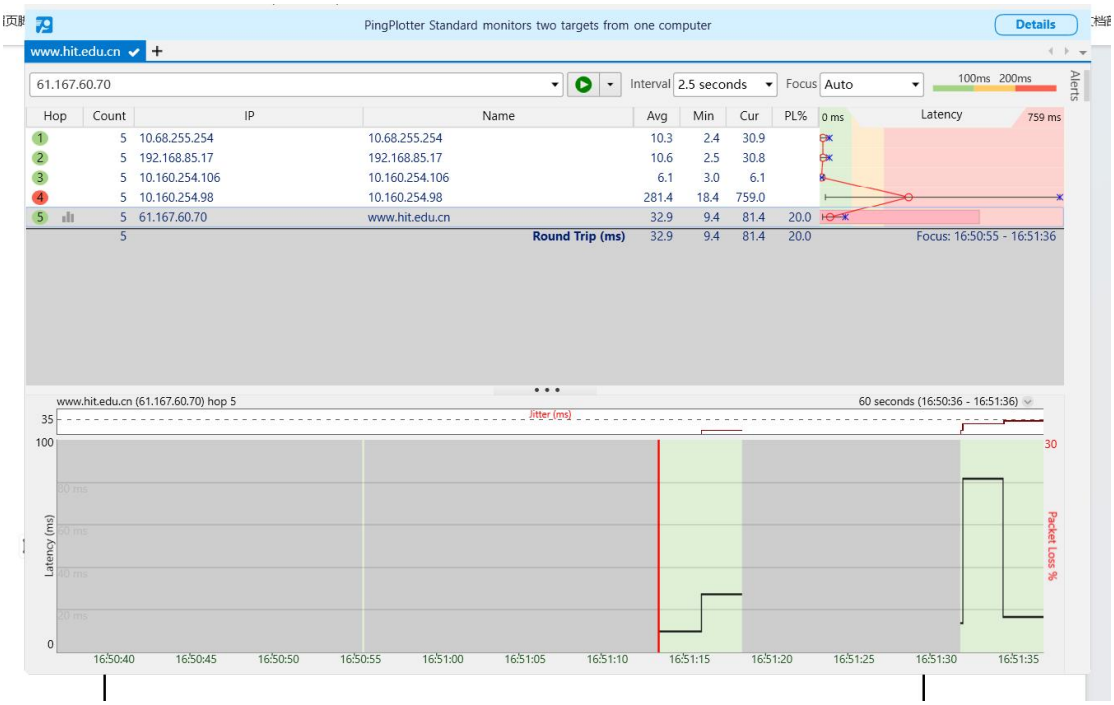
$$throughput = \frac{152932B}{(2.687326-1.771709)s} \approx 1.67\text{MB/s}$$

（四）IP 分析

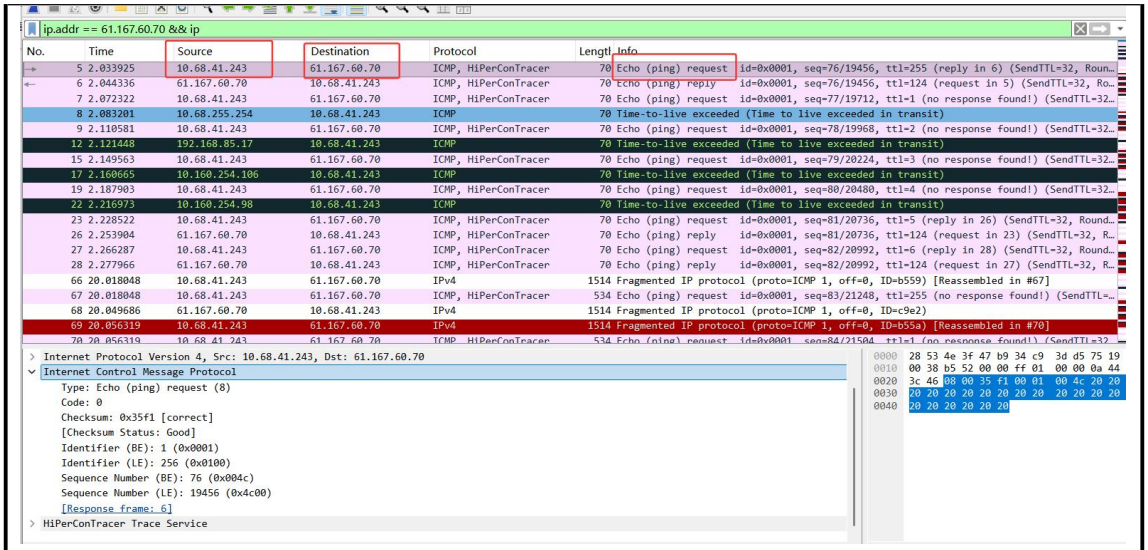
下载安装 PingPlotter 后安装打开。

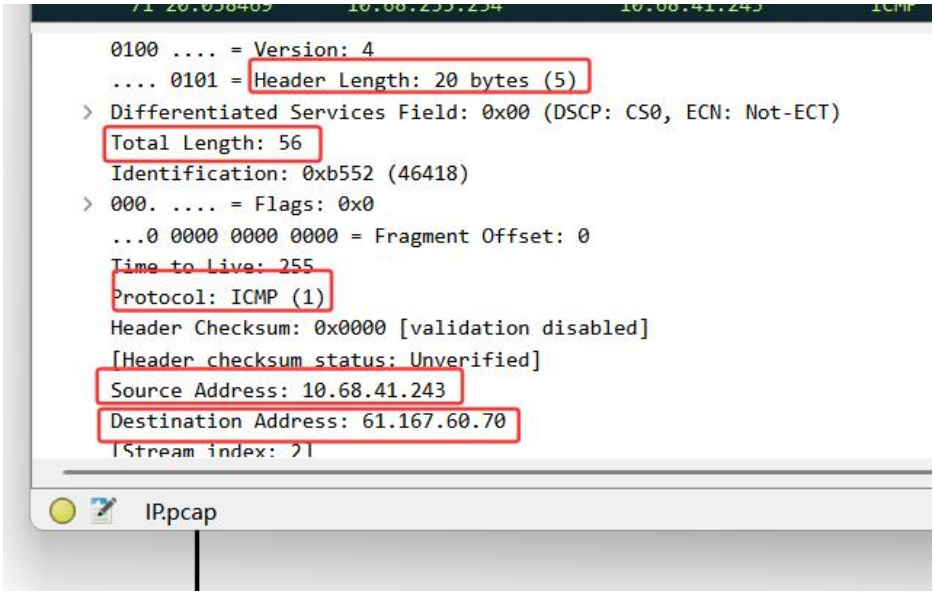


在 Edit->Options->Default Settings->Engine 界面打开查看发送的配置，报文类型为 ICMP Using Windows DLL，等待时长为 3s，时延设置为自动、包大小为 56B，并在开始后改为 2000B、3500B



发送界面如上图所示。





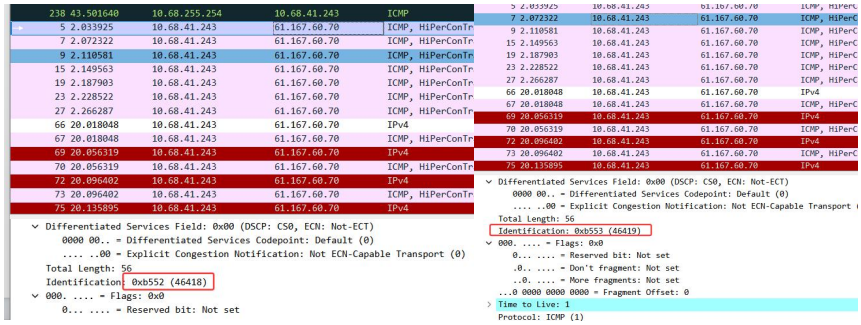
本地主机的 IP 地址是 10.68.41.243，服务器的 IP 地址是 61.167.60.70。IP 数据包头中，上层协议的字段值是 1，此处表示的是 ICMP 协议。 IP 头的总长度（Total Length 字段）是 56B，其中 IP 数据包的头部(Header Length 字段)是 20B，所以真正的净载字节的大小为 56B-20B=36B。 该数据包没有分片，可以查看 Flags 标志中的 DF（don't fragment）标志以及 MF（more fragments）标志为 0，标志该数据包可以被分片，并且没有被分片，或者该数据包为最后一片，且 Fragment Offset 片位移为 0，所以分组是没有被分片的

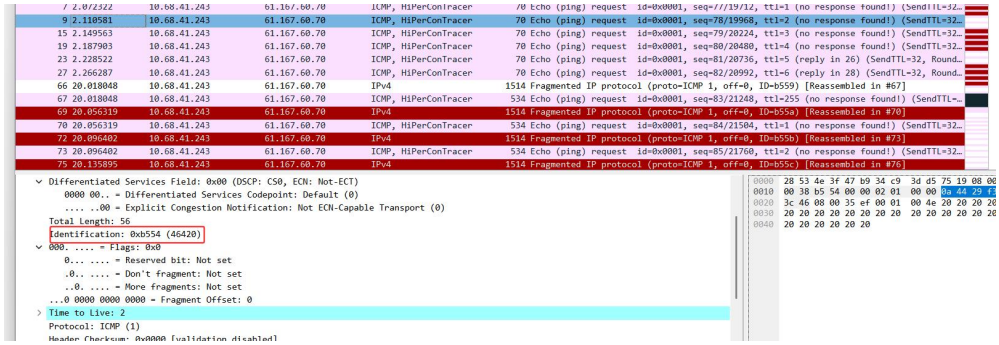
```
   .0.. .... = Don't fragment: Not set
   ..0. .... = More fragments: Not set
...0 0000 0000 0000 = Fragment Offset: 0
```

主机发出的 ICMP 中的 IP 数据包的 identification、TTL、Header Checksum 字段总是在改变。而 IP 数据报中的版本、首部长度、标志位、协议、源 IP 地址和目的 IP 地址等字段总保持不变。这样做是为了保证数据包能被正确解析、路由，并确保通信的完整性和可靠性。如果这些关键字段在传输过程中发生变化，可能会导致数据包无法正常送达或被错误处理。因此这些字段都应当保持一致。

TTL 每经过一个路由器递减 1，设置 TTL 的目的时防止数据包无限在网络内循环。

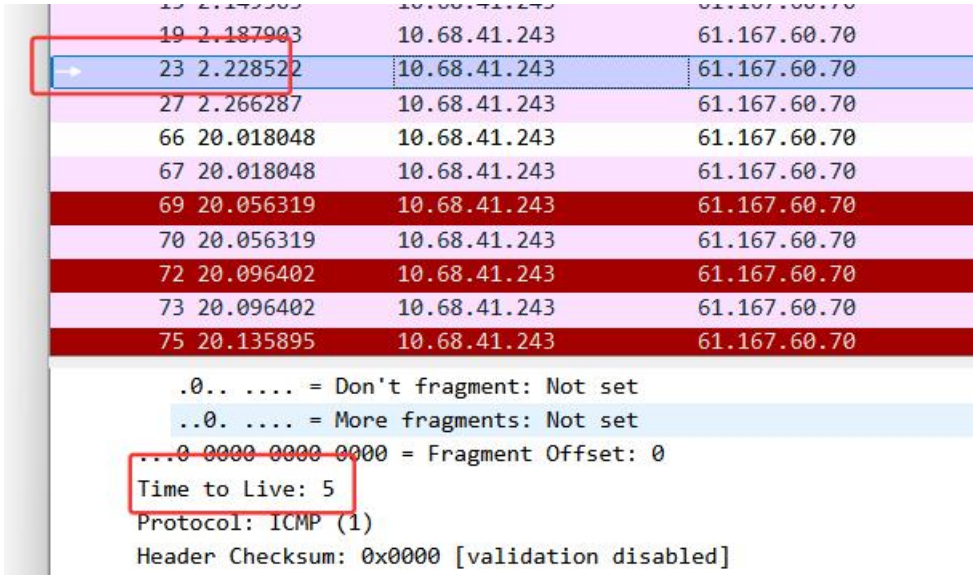
Identification 用于标识和重组分片所以每次发送新的数据包时递增变化。Header





IP 数据包中的 Identification 字段是以 16 进制数的形式出现的，并且在相邻的两个两个 IP 数据包中，这个字段是逐步的加一的递增的。

Checksum 变化是因为头部字段变化后，需要重新计算校验和。图中可以看出 TTL 从 1-4 都没有正确回复，当 TTL 为 5 时才有正确回复以及帧序号。



下页图中是最近的第一条路由器返回的 ICMP 的 TTL exceeded 消息。

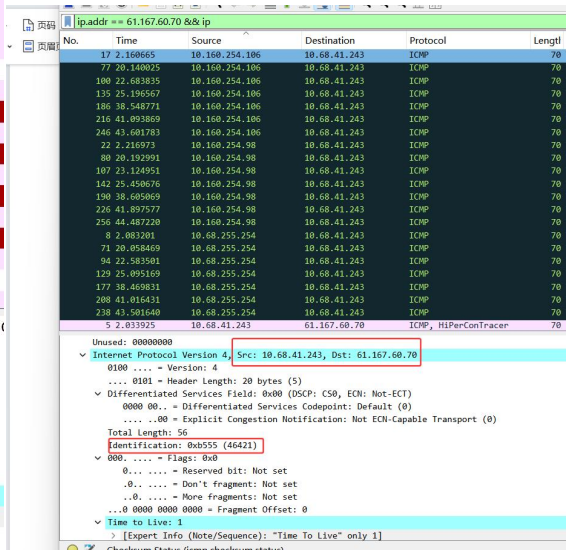
Identification 字段是 0x005e（94），TTL 字段是 253。 Identification 字段是会改变的，因为 Identification（标识符）用于在 IP 分片时标识属于同一数据包的所有片段，确保接收方可以正确重组它们。如果数据报没有分片要求，Identification 仍会分配一个值，作为该数据包的唯一标识。

No.	Time	Source	Destination	Protocol	Length	Info
17	2.160665	10.160.254.106	10.68.41.243	ICMP	70	Time-to-li
77	20.140025	10.160.254.106	10.68.41.243	ICMP	70	Time-to-li
100	22.683835	10.160.254.106	10.68.41.243	ICMP	70	Time-to-li
135	25.196567	10.160.254.106	10.68.41.243	ICMP	70	Time-to-li
186	38.548771	10.160.254.106	10.68.41.243	ICMP	70	Time-to-li
216	41.093869	10.160.254.106	10.68.41.243	ICMP	70	Time-to-li
246	43.601783	10.160.254.106	10.68.41.243	ICMP	70	Time-to-li
22	2.216973	10.160.254.98	10.68.41.243	ICMP	70	Time-to-li
80	20.192991	10.160.254.98	10.68.41.243	ICMP	70	Time-to-li
107	23.124951	10.160.254.98	10.68.41.243	ICMP	70	Time-to-li
142	25.450676	10.160.254.98	10.68.41.243	ICMP	70	Time-to-li
190	38.605069	10.160.254.98	10.68.41.243	ICMP	70	Time-to-li
226	41.897577	10.160.254.98	10.68.41.243	ICMP	70	Time-to-li
256	44.487220	10.160.254.98	10.68.41.243	ICMP	70	Time-to-li
8	2.083201	10.68.255.254	10.68.41.243	ICMP	70	Time-to-li
71	20.058469	10.68.255.254	10.68.41.243	ICMP	70	Time-to-li
94	22.583501	10.68.255.254	10.68.41.243	ICMP	70	Time-to-li
129	25.095169	10.68.255.254	10.68.41.243	ICMP	70	Time-to-li
177	38.469831	10.68.255.254	10.68.41.243	ICMP	70	Time-to-li
208	41.016431	10.68.255.254	10.68.41.243	ICMP	70	Time-to-li
238	43.501640	10.68.255.254	10.68.41.243	ICMP	70	Time-to-li
5	2.033925	10.68.41.243	61.167.60.70	ICMP, HiPerConTracer	70	Echo (ping

```
    Identification: 0x005e (94)
  ⌄ 000. .... = Flags: 0x0
      0... .... = Reserved bit: Not set
      .0.. .... = Don't fragment: Not set
      ..0. .... = More fragments: Not set
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 253
    Protocol: ICMP (1)
    Header Checksum: 0x7f65 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 10.160.254.106
    Destination Address: 10.68.41.243
    [Stream index: 7]
⌄ Internet Control Message Protocol
  ⌄ Type: Time-to-live exceeded (11)
```

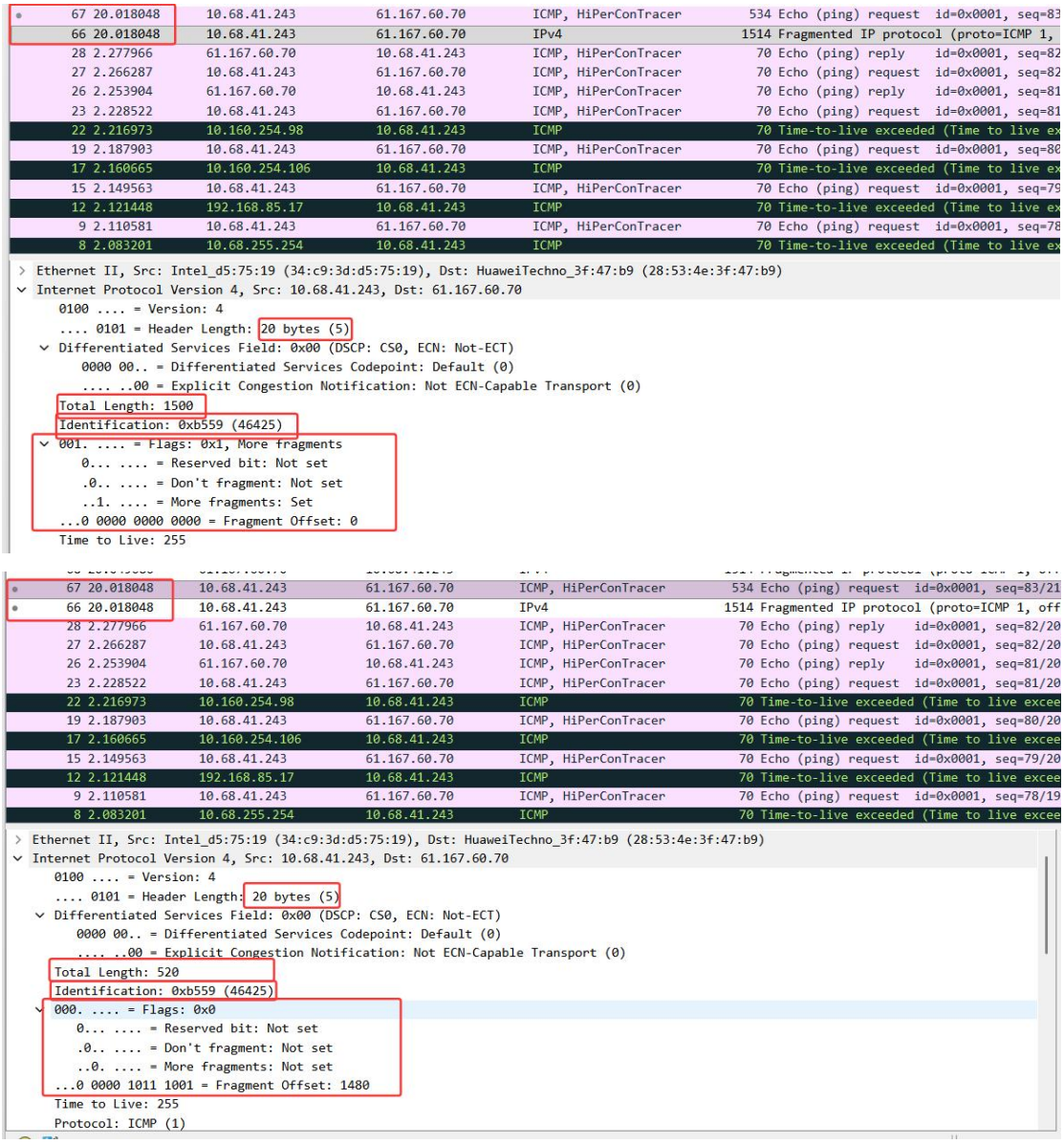
15	2.149563	10.68.41.243	61.167.60.70
19	2.187903	10.68.41.243	61.167.60.70
23	2.228522	10.68.41.243	61.167.60.70
27	2.266287	10.68.41.243	61.167.60.70
66	20.018048	10.68.41.243	61.167.60.70
67	20.018048	10.68.41.243	61.167.60.70
69	20.056319	10.68.41.243	61.167.60.70
70	20.056319	10.68.41.243	61.167.60.70
72	20.096402	10.68.41.243	61.167.60.70
73	20.096402	10.68.41.243	61.167.60.70
75	20.135895	10.68.41.243	61.167.60.70
76	20.135895	10.68.41.243	61.167.60.70
78	20.174887	10.68.41.243	61.167.60.70
79	20.174887	10.68.41.243	61.167.60.70
82	20.212233	10.68.41.243	61.167.60.70
83	20.212233	10.68.41.243	61.167.60.70

```
      .... ..00 = Explicit Congestion Notification: Not ECN-(
    Total Length: 56
    Identification: 0xb555 (46421)
  ⌄ 000. .... = Flags: 0x0
      0... .... = Reserved bit: Not set
      .0.. .... = Don't fragment: Not set
      ..0. .... = More fragments: Not set
    ...0 0000 0000 0000 = Fragment Offset: 0
  ⌄ Time to Live: 3
    › [Expert Info (Note/Sequence): "Time To Live" only 3]
    Protocol: ICMP (1)
```



可以看到在第一封 ICMP 数据包中包含了 TTL 为 1 的失效报文的内容，它与主机发送的 IP 同一标识符的数据包内容相对应。由图可知，Identification 未变化，而 TTL 由于发生跳转而

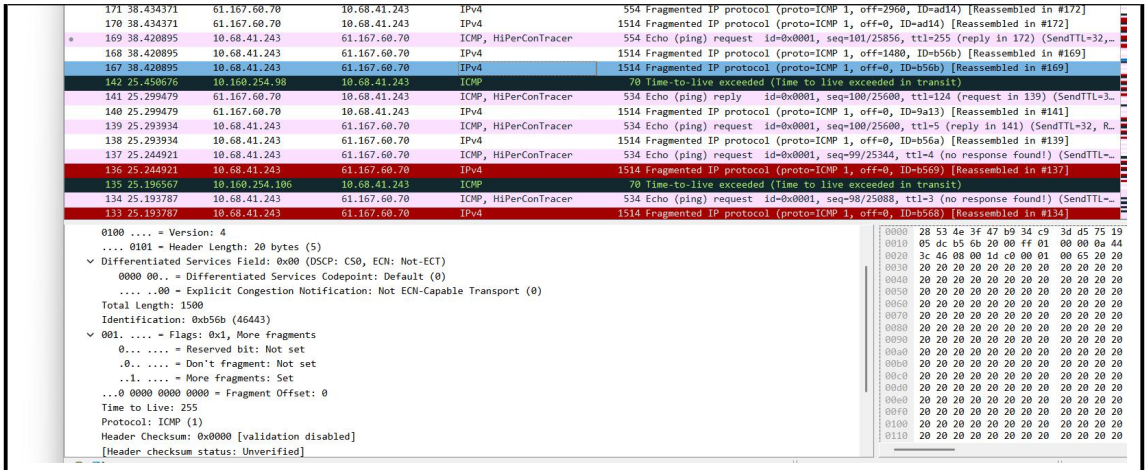
减小。



按时间排序后，找到数据包大小发生明显变化的位置，从而迅速定位发生改变大小后的数据包位置。

从截图可以看出，数据包被分解成两个。在第一个 IP 数据包中，DF 位被置为 0，MF 位被置为 1，说明后续还有其他数据包。而偏移量为 0，说明为第一个数据包，其长度既可通过总长度减去首部长度，即 1500B - 20B = 1480B 得到。也可以通过第二个分片中的偏移量得到(Fragment Offset == 1480)。

根据计算，由于 MTU 为 1500，因此数据包将被分为 3 片，只要找到第一次连续两次出现 Length == 1514 的位置即可。在这些数据包中，由于位于数据中的位置不同，首先携带的数据内容肯定有所差别而首部中的偏移量(1:0，2:1480，3:2960)必然不同。第一片与第二片报文除此之外，由于都还有后续的分片所以标志位相同。相应的，由于最后一片报文没有后续报文，所以 MF 为被置 0，因此和第一片、第二篇报文存在差异。



（五）抓取 ARP 数据包

```
PS C:\Users\97173> arp -a

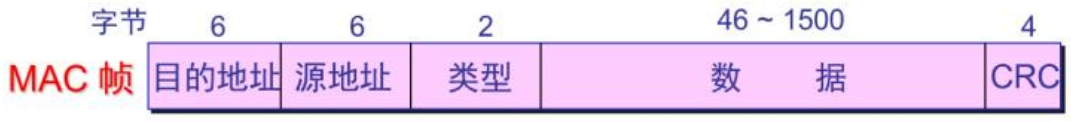
接口： 10.68.41.243 --- 0xb
  Internet 地址         物理地址              类型
  10.68.255.254         28-53-4e-3f-47-b9     动态
  224.0.0.22            01-00-5e-00-00-16     静态
  224.0.0.251           01-00-5e-00-00-fb     静态
  224.0.0.252           01-00-5e-00-00-fc     静态
  239.255.255.250       01-00-5e-7f-ff-fa     静态
  255.255.255.255       ff-ff-ff-ff-ff-ff     静态

接口： 192.168.142.1 --- 0x10
  Internet 地址         物理地址              类型
  192.168.142.254       00-50-56-e4-14-ef     动态
  192.168.142.255       ff-ff-ff-ff-ff-ff     静态
  224.0.0.22            01-00-5e-00-00-16     静态
  224.0.0.251           01-00-5e-00-00-fb     静态
  224.0.0.252           01-00-5e-00-00-fc     静态
  239.255.255.250       01-00-5e-7f-ff-fa     静态
  255.255.255.255       ff-ff-ff-ff-ff-ff     静态

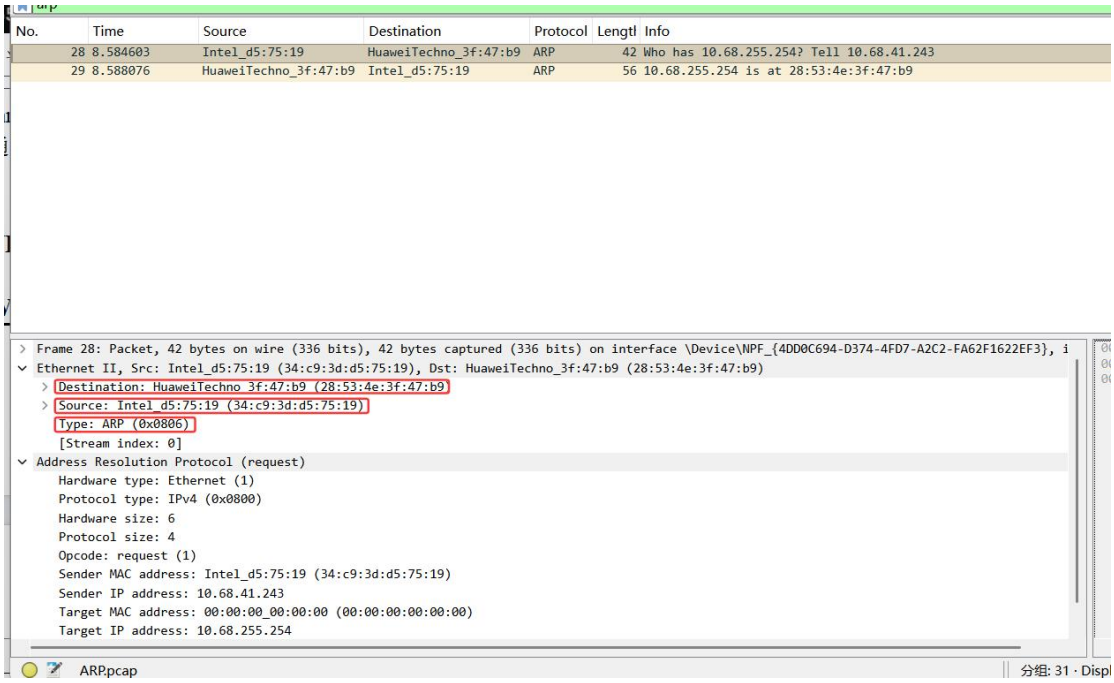
接口： 192.168.136.1 --- 0x16
  Internet 地址         物理地址              类型
  192.168.136.254       00-50-56-e7-e6-f1     动态
  192.168.136.255       ff-ff-ff-ff-ff-ff     静态
  224.0.0.22            01-00-5e-00-00-16     静态
  224.0.0.251           01-00-5e-00-00-fb     静态
  224.0.0.252           01-00-5e-00-00-fc     静态
  239.255.255.250       01-00-5e-7f-ff-fa     静态
  255.255.255.255       ff-ff-ff-ff-ff-ff     静态
```

输入 arp -a 以后显示了当前的 arp 缓存，如上图所示。ARP 缓存的第一列是当前局域网设备的 IP 地址，第二列是上述 IP 地址对应的 MAC 地址，以 16 进制表示。第三列表示 ARP 缓存的类型，dynamic 表示缓存项是通过 ARP 请求动态获取的，在一段时间后会过期，Static 表示缓存项是通过静态配置添加的，不会自动过期。

我们可以看到，Ethernet II 的首部结构是这样的。我们以此作为对照，在下页附图中，可以



看到，抓到的包中格式完全与上页末尾的图相对应。



先删除了本地 arp 缓存，由于 ping 192.168.1.82 失败，选择了 ping 61.167.60.70。

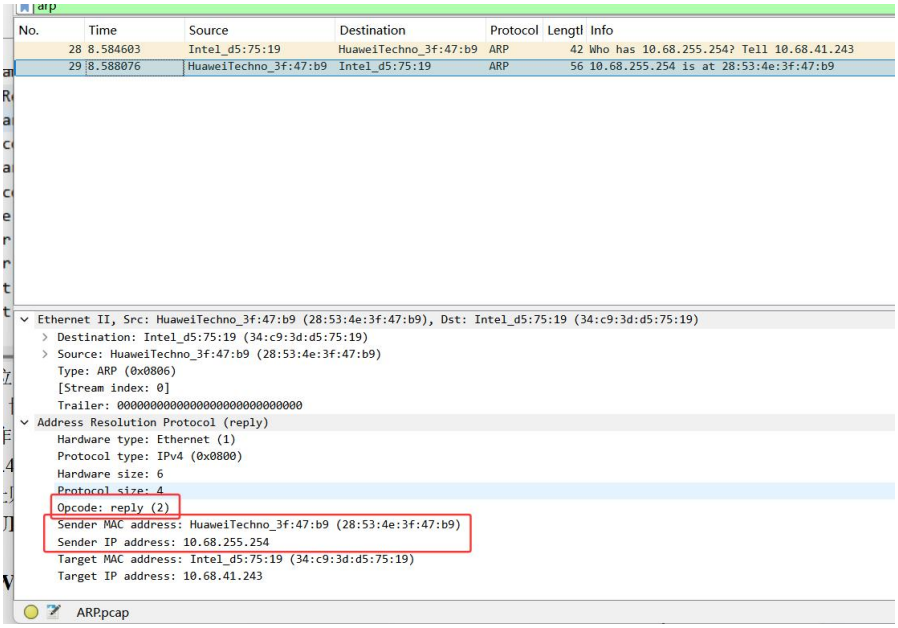




继续根据提供的 ARP 格式包分析数据。

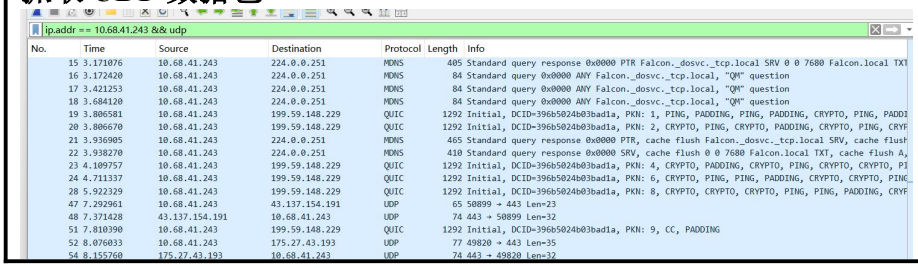
```
[Stream index: 0]
Address Resolution Protocol (request)
    Hardware type: Ethernet (1)
    Protocol type: IPv4 (0x0800)
    Hardware size: 6
    Protocol size: 4
    Opcode: request (1)
    Sender MAC address: Intel_d5:75:19 (34:c9:3d:d5:75:19)
    Sender IP address: 10.68.41.243
    Target MAC address: 00:00:00_00:00:00 (00:00:00:00:00:00)
    Target IP address: 10.68.255.254
```

可以看到第一位置为 1，表示硬件类型为以太网，运输层协议则为 IPv4(对应 0x0800)，硬件地址长度为 6，协议地址长度为 4，单位均为 1 字节。和我们在 Wireshark 抓包获取的长度完全一致。操作码置 1 表示为请求操作。依次为发送端 MAC 地址：34:c9:3d:d5:75:19，IP 地址则为10.68.41.243(与我们之前使用arp -a展示出的ARP缓存中WLAN设备的信息相符)，目标 MAC 地址则为 00:00:00:00:00:00，IP 地址为 10.68.255.254。在回应的报文中，我们可以看到 Opcode 置为 2，表示 reply(因为发送方的 IP 地址与我们先前请求的 IP 地址相吻合，而且目标 MAC 地址与目标 IP 地址也与本地设备一致)。

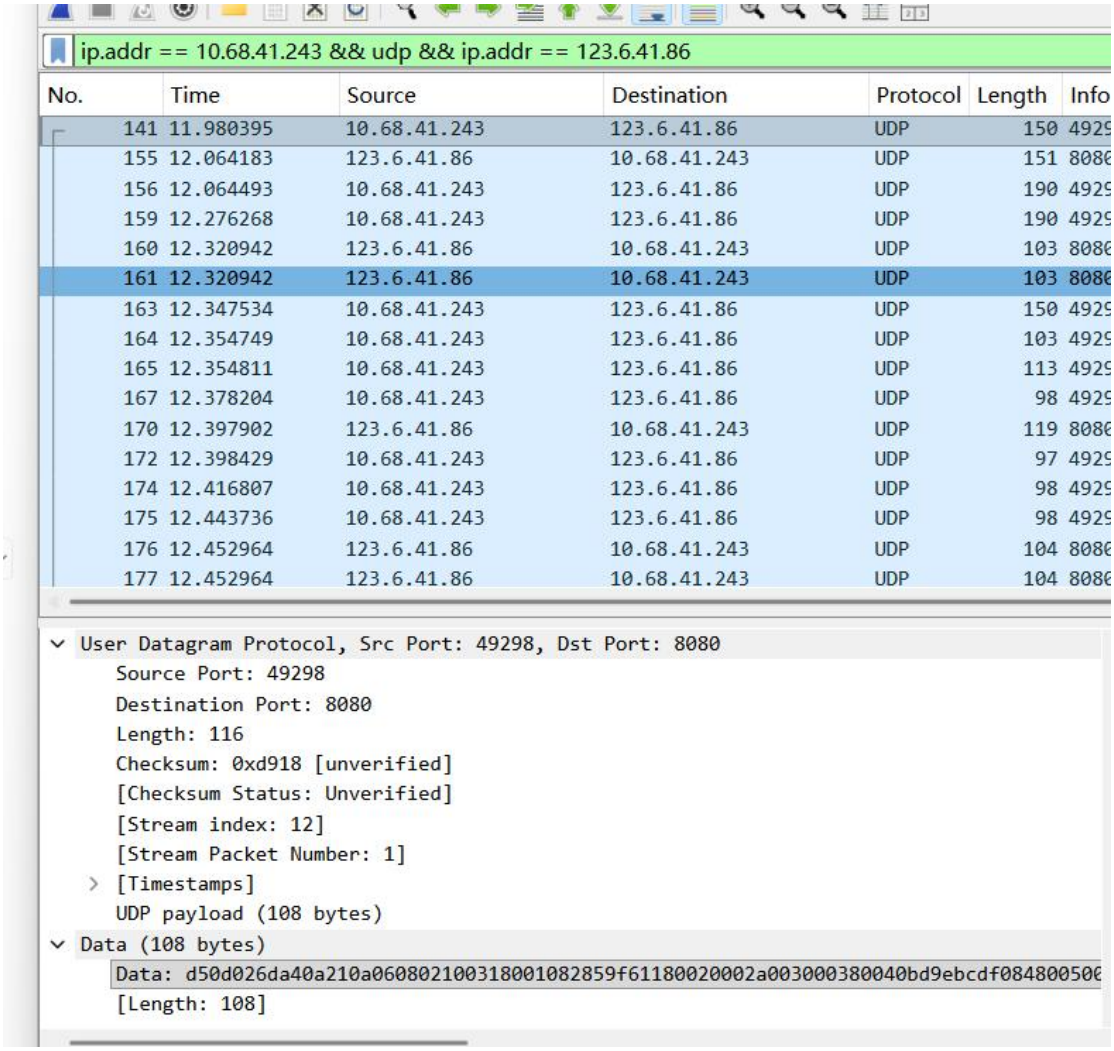


对于发送方主机来说，它只知道目标的 IP 地址，而不知道目标的 MAC 地址，所以只能通过广播 MAC 地址向所有的设备发送 ARP 请求，能够确保所有设备都可以接受到请求，而通过在 ARP 报文中添加已知的目的 IP 地址又可以保证只有 IP 匹配的设备才会发出响应。而 ARP 响应的时候，由于 ARP 查询过程中包括了查询主机的 IP 地址和 MAC 地址，可以之间将应答确切的发送给主机，不需要进行广播过程。

抓取 UDP 数据包



先进行数据报的过滤，输入 IP 地址为本地主机地址和协议类型为 UDP 进行基本的过滤。因为语音通话作为流媒体实时通话，它对最低速率有要求而且允许一部分数据丢失，因此更可能选择 UDP 协议来传输，而不是选择时延较长但是更可靠的 TCP 协议传输数据。经筛选过滤后，可以看到还存在着许多其他协议。但是由于微信语音通话功能明确使用了 UDP 协议进行传输，因此我们先专心分析 UDP 数据包。查看存在的 UDP 数据包后可以发现，主要有两个 IP 地址 123.6.41.86 和 111.55.114.67。由于后者仅作为从本机发送的 UDP 报文的接收方存在，而且经查询资料显示，123.6.41.86 主机固定使用了 8080 端口，因此可能是微信服务器对应的 IP 地址(http://www.blogjava.net/yongboy/archive/2014/03/05/410636.html)，所以选择观察 123.6.41.86 对应的报文。

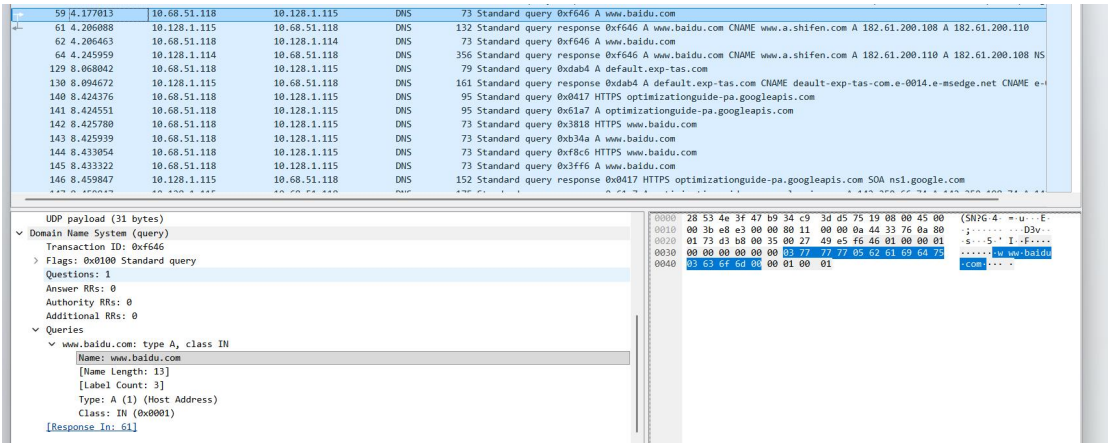


在上图中可以清楚看到，UDP 首部中包含了本机使用的端口号(即 49298)，占 4 字节；对方使用的端口号为 8080，占 4 字节；数据长度为 116，占 4 字节；校验和为 0xd918，占 4 字节。以及流地址、流数据包的序号和时间戳。

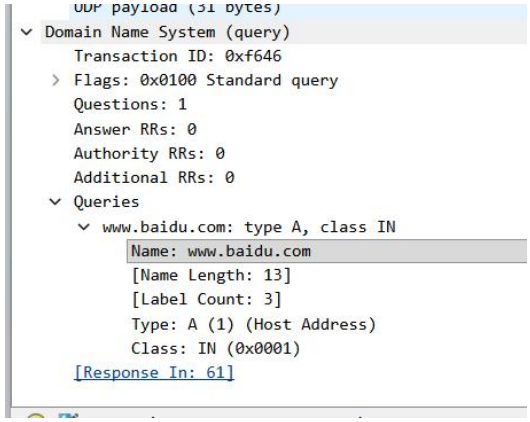
在发送应用数据之前，TCP 必须先进行三次握手（SYN, SYN-ACK, ACK）来协商序列号、窗口大小等参数，确保双方都准备好通信，确认线路通畅后再开始正式交谈，这就相当于在逻辑上建立了一个连接并不断维护着这个连接的状态。UDP 没有任何前置流程。它想发送数据时，直接构造一个数据包（数据报）并发送出去，不关心目标是否在线、是否准备好接收。而且，TCP 根据可靠数据传输理论，通过维护逻辑连接状态和差错检验机制，确保了数据一定可以以正确的顺序、正确的内容交付给上层；而 UDP 协议由于不存在连接，因此

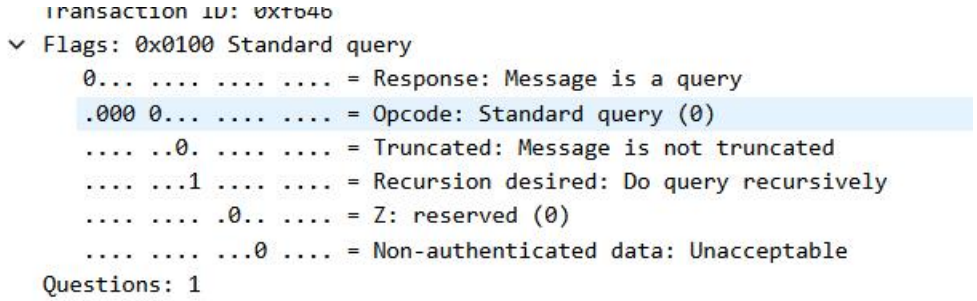
不能接收到反馈机制，不会去进行重传、按序交付，因此 UDP 协议是不可靠的。

（六）利用 Wireshark 进行 DNS 分析



主机想要访问域名时，首先向本地域名服务器查询是否存储该域名对应 IP 的映射，若本地域名服务器中没有缓存，则由本地域名服务器继续查询，分为递归和迭代两种方式，本地域名服务器首先查询根域名服务器，由根域名服务器返回顶级域名服务器的 IP 地址，再继续查询。可以看到 DNS 协议的报文中，包含了事务 ID、标志位、询问数量、Answer RRs(包含与你所查询的域名和记录类型完全匹配的资源记录) 、Authority RRs (指向一个或多个负责该域名及其所在区域的权威域名服务器，包含所查询域名的 NS 记录。当递归 DNS 服务器本地缓存中没有答案时，它需要从根域名开始逐级查询。如果被查询的服务器自己不是该域名的权威服务器，但它知道该去找谁，它就会在 Authority RRs 中指明权威服务器是谁)、 Additional RRs (提供与本次查询相关的、可能有用的额外信息，通常是为了避免客户端需要再次发起额外的查询请求，最常见的是提供在 Authority RRs 中列出的那些权威域名服务器所对应的 IP 地址)。在标志位中，可以看到，第一位表示报文是否为回应报文(为 1 表





示是)，2~5 位表示操作码，为 0 表示为标准询问操作。第 6 位置 1 表示报文因过长被截断，当 DNS 响应报文大小超过 512 字节时，服务器会设置 TC(Truncated)标志位。第 7 位置 1 表示指示服务器应进行递归查询，第 8 位 Z 位为保留位，留给以后使用。最后一位仅在响应报文中有效，1 表示响应服务器是所查询域名的权威名称服务器，0 表示响应来自缓存或非权威服务器。

实验结果：

（一）Wireshark 的使用

根据实验指导书，我们利用工大官网完成了对Wireshark基本操作的学习与使用。了解了命令菜单、俘获分组列表、分组头部明细、分组内容窗口、显示筛选规则这几个基本板块的位置和使用方式。

（二）HTTP 分析

1）HTTP GET/response 交互

由之前的抓包分析与截图可知，http://hitgs.hit.edu.cn/news 网页不存在，浏览器使用的是 HTTP 1.1 协议，接收语言为简体中文。而在 IP 层面，使用 IPv6 协议，目的 IP 地址为 2001:da8:b800:253::dbd9:e219。本机 IP 地址为 2406:280:1008:388e:cda4:f1b2:dab7:6884。

2）HTTP 条件 GET/response 交互

两次访问 http://hitgs.hit.edu.cn/news，HTTP报文并不会添加If-modified-since行。

换为其他正常网页如 http://example.com，则可以观察到。

（三）TCP 分析

由前面的分析可知，128.199.245.12 即为 gaia.cs.umass.edu 服务器的 IP 地址，进一步从 info 中可见发送端口号为 2845 或者 14124，接收端口号为 80。结合实际情况分析了三次握手的过程以及数据传输速率

（四）IP 分析

IP 数据包中的 Identification 字段是以 16 进制数的形式出现的，并且在相邻的两个两个 IP 数据包中，这个字段是逐步的加一的递增的。

Checksum 变化是因为头部字段变化后，需要重新计算校验和。图中可以看出 TTL 从 1-4 都没有正确回复，当 TTL 为 5 时才有正确回复以及帧序号。

本地主机的 IP 地址是 10.68.41.243，服务器的 IP 地址是 61.167.60.70。IP 数据包头中，上层协议的字段值是 1，此处表示的是 ICMP 协议。IP 头的总长度由 Total Length 字段表明，其中 IP 数据包的头部(Header Length 字段)是 20B。数据包有没有分片，可以查看 Flags 标志中的 DF（don't fragment）标志以及 MF（more fragments）标志为 0，标志该数据包可以被分片，并且没有被分片，或者该数据包为最后一片。

主机发出的 ICMP 中的 IP 数据包的 identification、TTL、Header Checksum 字段总是在改变。而 IP 数据报中的版本、首部长度、标志位、协议、源 IP 地址和目的 IP 地址等字段总保持不变。这样做是为了保证数据包能被正确解析、路由，并确保通信的完整性和可靠性。如果这些关键字段在传输过程中发生变化，可能会导致数据包无法正常送达或被错误处理。因此这些字段都应当保持一致。

（五）抓取 ARP 数据包

ARP 协议格式简单且明确。本机 ARP 报文第一位置为 1，表示硬件类型为以太网，运输层协议则为 IPv4(对应 0x0800)，硬件地址长度为 6，协议地址长度为 4，单位均为 1 字节。操

作码置 1 表示为请求操作。依次为发送端 MAC 地址：34:c9:3d:d5:75:19，IP 地址则为 10.68.41.243，目标 MAC 地址则为 00:00:00:00:00:00(广播地址)，IP 地址为 10.68.255.254。在回应的报文中，我们可以看到 Opcode 置为 2，表示回复。

（六）抓取 UDP 数据包

微信语音通话使用了多种协议进行和维护通讯，UDP首部中包含了本机使用的端口号(即49298)，占4字节；对方使用的端口号为8080，占4字节；数据长度为116，占4字节；校验和为0xd918，占4字节。以及流地址、流数据包的序号和时间戳。

（七）利用 WireShark 进行 DNS 协议分析

详细分析了DNS协议的报文结构

问题讨论：

在 HTTP 条件 GET/response 交互部分中，重新访问 http://hitgs.hit.edu.cn/news 并不会添加 If-modified-since 行。这是因为 302 是临时重定向，告诉客户端资源暂时在另一个位置，客户端收到 302 后，会向新的 URL 发起全新的请求，这个新请求被视为独立的请求，与原始请求无关。客户端认为原始 URL 仍然是规范 URL，不会将缓存状态转移到新位置。缓存技术是基于完整的 URL（包括查询参数），`/old-location` 和 `/new-location` 是不同的缓存键，客户端没有 `/new-location` 的缓存，自然不会发送条件请求头。因此每次请求都会需要重新下载资源。

在 UDP 数据包抓取中，经筛选过滤后，可以看到还存在着许多其他协议如 QUIC、mDNS、RTCP、NTP 这些协议。其中 mDNS 协议是一种无需配置 DNS 服务器即可进行名称解析的协议。 它使主机能够在同一虚拟局域网（VLAN）内无缝地进行通信，所以猜测其用来进行 P2P 传输，mDNS 使用多播数据包来发现网络中的不同主机。NTP（Network Time Protocol）为网络时间协议，是一个应用层协议，用来使客户端和服务器之间进行时钟同步，提供高精准度的时间校正，猜测用来校准通话时间。NTCP 协议是实时传输控制协议（Real-time Transport Control Protocol 或 RTP Control Protocol 或简写 RTCP），是实时传输协议（RTP）的一个姐妹协议。RTP 使用一个偶数 UDP port；而 RTCP 则使用 RTP 的下一个 port，也就是一个奇数 port。RTCP 为 RTP 媒体流提供信道外（out-of-band）控制。RTCP 本身并不传输数据，但和 RTP 一起协作将多媒体数据打包和发送。RTCP 定期在流多媒体会话参加者之间传输控制数据。RTCP 的主要功能是收集相关媒体连接的统计信息并为 RTP 所提供的服务质量（Quality of Service）提供反馈，猜测其是用来对语音通话质量进行检测。由此可见，微信语音通话是一个多协议参与调控的复杂过程。

根据查阅资料表明微信语音通话具体是采用点对点（P2P）直连还是经由服务器中转，并不是固定的，它会根据实际的网络状况动态选择，目的是在延迟、畅通率和成本之间取得平衡。值得注意的是，微信在实际通话中可能会采用混合模式。例如，有测试发现，在同一 WiFi 下的两台设备，通话时音频和视频流可能分别采用不同的传输路径（一部分走 P2P，另一部分走中转），以此兼顾流畅性和可靠性。所以可能这个过程中出现的两个比较常见的 IP 地址一个为服务器地址，另一个则为对方的 IP 地址。

心得体会：

在完成本次实验的过程中，我深刻体会到网络协议的存在。通过利用Wireshark捕获并解析数据包，我直观地看到了HTTP、TCP、IP等协议在实际网络环境中的运作方式，从而加深了我对计算机网络体系结构的理解。在分析HTTP协议时，我不仅观察到了浏览器与服务器之间请求与响应的完整交互过程，还通过对比首次访问与条件GET请求的差异，理解了缓存机制在Web性能优化中的作用。TCP协议的分析让我对可靠数据传输有了更具体的认识。通过追踪三次握手、数据传输和滑动窗口的变化，我看到了序列号、确认机制和流量控制是如何协同工作以保证数据完整性和传输效率的。特别是在观察TCP连接建立和终止过程中SYN、ACK、FIN标志变成了真实的数据包字段，这种将理论映射到实践的过程让我对TCP的可靠性机制有了更深层次的理解。IP协议分析环节让我领略了网络层的重要性。通过使用PingPlotter发送IP数据包观察TTL值的变化和IP分片现象，我直观地看到了数据包在网络中跳转的路径，理解了路由器如何处理超过MTU的数据包。看到同一个原始数据包被分成多个片段传输，然后在目的地重组，这种“化整为零、聚零为整”的过程让我对IP协议的适应性和灵活性印象深刻。在UDP协议分析环节，通过捕获微信语音通话的数据包来观察UDP的实际应用。与TCP形成鲜明对比的是，UDP数据包简洁的头部结构和缺乏确认机制的特点非常明显。在语音通话这种实时性要求高的场景中，UDP以其低延迟的特性展现出独特优势，但同时我也看到了可能存在的丢包问题，这让我理解了为什么不同的应用场景需要选择不同的传输层协议，同时也看到了微信语音聊天实际的复杂过程，感受到了计算机应用的工程和实际教学的差距。整个实验过程中，Wireshark这个强大的工具让我能够“透视”网络通信的本质。从最初的不知所措到后来能够熟练设置过滤条件、解读各层协议头部字段，不仅掌握了协议分析的基本技能。